F-PABEE: Flexible-patience-based Early Exit for Single-label and Multi-label text Classification Tasks

Anonymous ACL submission

Abstract

In recent years, pre-training language models(PLMs) such as BERT and ALBERT have achieved high accuracy in many single-label and multi-label classification tasks. However, their industrial usages are limited by their large inference latency. In order to increase the inference speed of BERT while maintaining good performances, this paper proposes a Flexible-patience-based early exiting (F-PABEE) method. F-PABEE makes predictions at each layer's intermediate classifier and will exit early if the current layer and the last few layers have similar (similarity score less than a threshold) predicted distributions. Our method is more flexible than the previous state-of-the-art early exiting method PABEE since it can achieve different speed-performance trade-offs by adjusting the similarity score threshold and the patience parameter. We conduct a wide range of experiments on single-label classification tasks from the GLUE benchmark and four multi-label classification tasks and find that: (a) F-PABEE outperforms existing early exit models such as Brexit and PABEE on both single-label and multi-label tasks; (b) F-PABEE performs well on different PLMs such as ALBERT; (c) ablation studies show that as a similarity measure, Jenson-Shannon divergence works best for our F-PABEE method.[1]

1 Introduction

In recent years, fine-tuning pre-trained language models (PLMs) have become the standard solution for almost all Natural Language Processing tasks (Lin et al., 2021).Despite their state-of-the-art (SOTA) performances, BERT (Devlin et al., 2018) and its variants (Lan et al., 2019; Yang et al., 2019; Radford et al., 2019; Liu et al., 2019) still face a significant challenge in applications: high latency due to their large parameter numbers and model depth. Generally, pre-training models are mainly based on dozens of stacked transformers, and higher-accuracy models usually rely on deeper transformer architectures, which may require higher computational resources, capital, and time costs. In order to speed up the inference of PLMs, a series of methods have been proposed, such as attention head pruning, layer pruning, knowledge distillation, and input-adaptive inference methods. Early exiting, as an essential input-adaptive inference method, has attracted much attention.

Early exiting installs a classifier (or exit) at each transformer layer of BERT and evaluates at each layer whether the current layer's prediction is suited to be the final prediction. Thus, it can process different samples with different complexity and, as a result, strike a balance between model performance and speedup ratios. The core of early exiting is to design a strategy to determine the quality of the current intermediate layer's prediction. There are mainly two types of early exiting strategies. The first one is to evaluate the current layer's prediction based on specific confidence measurements, such as entropy BranchyNet (Teerapittayanon et al., 2016) maximum probability mass Shallow-Deep (Kaya and Dumitras, 2018). Recently, Berxit (Xin et al., 2021) asks the model to learn to evaluate the confidence of intermediate layers' predictions. The second type of early exiting strategies is represented by PABEE (Zhou et al.). PABEE evaluates the quality of the prediction by comparing it with the previous layers', mimicking the idea of model early stopping.

We raise two issues for the current early exiting literature. Despite its SOTA performances, PABEE still faces a limitation for application: it can not flexibly adjust the speedup ratio on a given task and a fixed patience parameter, mainly caused by PABEE's strict cross-layer comparison strategy. Thus, we wonder whether we can combine PABEE with a softer cross-layer comparison strategy. Second, the current early exiting literature mainly fo-

[1]Code will be made publicly available upon acceptance.

cuses on the single-label classification (SLC) tasks, and multi-label classification (MLC) tasks are neglected. So can the current SOTA methods like Berxit and PABEE speed up MLC tasks with good performances?

This paper proposes a Flexible-patience-based early exiting (F-PABEE) method to address the above issues. F-PABEE makes predictions at each intermediate layer's exit and will exit early if the current layer and the last few layers have similar (similarity score less than a threshold) predicted distributions. The similarity of predicted probability distributions from the current and previous layers are measured by divergence-based metrics like Kullback-Leibler Divergence (KL-divergence) and Jensen–Shannon divergence (JSD-dvergence). Our method can be seen as a natural extension of the PABEE method and is more flexible than PABEE since it can achieve different speed-performance tradeoffs by adjusting the similarity score threshold and the patience parameter. Our novel early exiting strategy can effortlessly extend to MLC tasks. An MLC task is usually regarded as a collection of binary classification tasks; thus, comparing the predictions across layers will summate similarity scores for binary predicted distributions.

We conduct extensive experiments on SLC tasks from the GLUE benchmark and four MLC tasks: (Yang et al., 2018; Hemphill et al., 1990; Coucke et al., 2018). The experimental results show that our F-PABEE method outperforms the existing SOTA early exiting methods on both SLC and MLC tasks. Ablation studies show that: (a) F-PABEE performs well on PLM backbones other than BERT; (c) as a similarity measure, Jenson-Shannon divergence works best for our F-PABEE method.

Our contributions are three-fold: (1) We propose F-PABEE, a novel and effective inference mechanism which is also flexible in adjusting the speedup ratios of BERT. (2) Our empirical results on the GLUE and four benchmark MLC tasks show that our method can effectively speed up inference across different tasks while maintaining good performances. (3) As far as we know, we are the first to investigate the early exiting of MLC tasks, and our F-PABEE is suitable for this type of task.

2 Related Works

Existing BERT inference speedup methods mainly include two categories: (1) Static approaches that compress the heavy models into a fixed smaller one. (2) Adaptive inference approaches that make no changes to the architecture and encode different samples with different parts of the model. Usually they can adjust certain hyper-parameters to dynamic controlling the average latency in product environments.

2.1 Static inference approach

Since the rise of BERT, there are a quite large numbers of literature devoting themselves in speeding up the inference of BERT. HeadPrune (Michel et al., 2019) propose to rank the attention heads and progressively pruning them, so that the inference latency is reduced. LayerDrop (Fan et al., 2019) further pre-trains and fine-tunes BERT with some of the layers skipped, so that we can reduce the layers on demand. DynaBERT (Hou et al., 2020) conduct pruning on attention heads, neurons in the feed-forward layers and layer depth. Knowledge distillation (KD) plays an important role in BERT model compression. (Sun et al., 2019) investigate the best practices of distilling knowledge from BERT into smaller-sized models. (Sanh et al., 2019) initialize the student model weights from the BERT teacher. (Jiao et al., 2019) applies attention KD, feature KD along with soft-label based KD, and introduce KD in the pre-training of student models. Based on lightweight integer-only approximation methods for nonlinear operations, I-BERT (Kim et al., 2021) performs an end-to-end integer-only BERT inference without any floating point calculation.

Note that despite some static model compression methods along can already speedup the BERT's inferences, they are still in the form of deep neural networks with multiple stacked layers. Thus, other inference speedup methods, like sample adaptive inference, can also be applied to further improve the efficiency.

2.2 Early exiting approaches

Recently, adaptive inference has attracted the attention of both academia and industries. One of the major methodology for adaptive inference is early exiting. It is a method to dynamically adjust certain hyper-parameters in response to changes in request traffic. Shallow transformer layers are used to infer simple requests, and deep transformer layers to infer complex requests. This method does not need to make significant changes to the original model structure or weight bits, nor does it need to

train different teacher-student learning networks, which saves computing resources. In addition, recent work point out deep transformers turn to be over-thinking, and early exiting could be beneficial for inference accuracy and generalization ability (Dehghani et al., 2018).

There are mainly three early exiting strategies for BERT dynamic exiting. The first one is score-based early exiting. BranchyNet (Teerapittayanon et al., 2016), FastBERT (Liu et al., 2020), and DeeBERT (Xin et al., 2020) calculated the entropy of the prediction probability distribution as an estimation for the confidence of exiting classifiers to enable dynamic early exiting. Shallow-Deep Nets (Kaya and Dumitras, 2018) and RightTool (Schwartz et al., 2020) leveraged the maximum of the predicted distribution as the exiting signal. The second type is the learned exiting (Elbayad et al., 2019). In this type of work, an early exiting signal is generated by a learnable module in the neural network. For example, BERxiT (Xin et al., 2021) install a fully connected layer right after each transformer block of BERT to output a score that is used to decide whether the BERT should stop inference and exit early. The third type is patience-based early exiting, which relies on cross-layer comparison to formulate the exiting signal. PABEE (Zhou et al.) propose a dynamic exiting strategy analogous to early stopping model training. That is, if the exits' predictions remain unchanged for a pre-defined number of times (patience), the model will stop inference and exit early. PABEE achieves SOTAs results for BERT early exiting.

This work complements the literature by proposing a more flexible extension to the PABEE method. Our method tends out to be performant in both SLC and MLC tasks. In addition, our method can flexibly adjust the speedup ratios to meet different industrial requirements.

3 Flexible patience-based early exiting

3.1 Motivation

Despite its SOTA performances, PABEE may suffer from two drawbacks. First, PABEE's exiting criterion is quite strict, since it asks the predicted labels of a few consecutive layers to be exactly the same. Thus, PABEE is inflexible in adjusting the speedup ratios. On a given task, once the multi-exit BERT is fine-tuned and the patience parameter is fixed, PABEE can only achieve a fixed average speedup ratio. Second, intuitively, PABEE acts like a student taking an exam. It checks the answer to a query again and again using the intermediate exits. He will be assured if the answers remain the same. However, what if the later intermediate exit find something wrong and want to change the answer? Thus, PABEE's strict exiting criterion may miss out some early exiting opportunities.

We also notice that the literature on early exiting mainly focuses on SLC tasks, and the investigations on whether the existing early exiting methods work well on MLC tasks are lacking. Intuitively, exit criteria of patience-based methods PABEE is too strict, and the exiting criteria on MLC tasks may be difficult to meet.

In summary, a more flexible early exiting mechanism should be designed, so that it is flexible than PABEE, and can perform well on both SCL tasks and MLC tasks.

3.2 Inference procedure for SLC task

We now introduce our flexible PABEE (F-PABEE) method. The architecture of F-PABEE is shown in Figure 1(b), which is an improved version of PABEE(Figure1(a)). Where L_i is the transformer blocks of the model, n is the number of transformer layers in the model, C_i is the inserted classifier layer, and S is the similarity score between adjacent classifier layers, $thre$ is the threshold, and pat is the pre-defined patience.

The inference process of the model is obvious; that is, the input sentences first pass embedded to the vector by the embedding layer:

$$h_0 = Embedding(x) \tag{1}$$

The vector is then passed through $L_1 ... L_n$ layers to extract features and compute its hidden state h, after which we use an internal classifier connected to each $L_1 \dots L_n$ layer predictors $C_1 ... C_n$ to predict probability p:

$$h_i = L_i(h_{i-1}); \tag{2}$$

$$p_i = C_i(h_i) \tag{3}$$

We denote the similarity score between the prediction results of layer i and layer $i+1$ as $s_{i,i+1} = S(p_i, p_{i+1})$ ($s_{i,i+1} \in \mathbf{R}$). The smaller the value of $s_{i,i+1}$, the prediction distributions p_i and p_{i+1} are more consistent with each other. The premise of the model's early exit is that the comparison scores between successive layers are relatively small; that

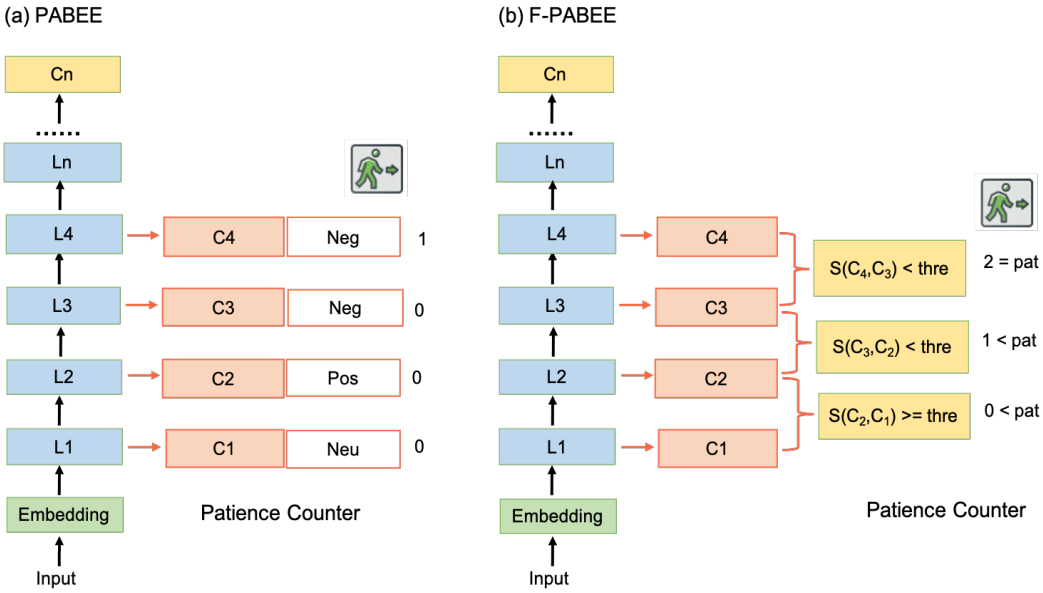

Figure 1: Architecture of PABEE and F-PABEE, C_i is classifier layer,S is score between adjacent layers, $thre$ is threshold, pat is patience in the model.

is to say, the output results of these layers are relatively similar, and the model is more confident in its prediction results. The threshold of the score τ is a hyper parameter. We use pat_i to store the number of times that the cross-layer comparison score is consecutively less than the threshold when the model reaches the current layer:

$$pat_{i+} = \begin{cases} pat_i + 1 & s_{i,i+1} < \tau \\ 0 & s_{i,i+1} >= \tau \end{cases} \quad (4)$$

Where τ is a predefined threshold, if $s_{i,i+1}$ is less than the predefined threshold, then increase the patience counter by 1. Otherwise, reset the patience counter to 0. This process is repeated until pat reaches the predefined patience value P_0. The model dynamically stops inference and exits early. However, if this condition is never met, the model uses the last layer of classifiers to make predictions. In this way, the model can stop inference early without going through all layers.

3.3 Similarity measures for SLC tasks

Note that our F-PABEE framework is flexible and general, wrapping the PABEE method as a particular case. In PABEE, the value of $s_{i,i+1}$ is 1 only when the labels predicted by two adjacent layers are the same; otherwise, it is 0.

Under the framework of F-PABEE, we can adopt different similarity measures for predicted probability distributions. In this work, we will compare a series of divergence based measurements:

- F-PABEE-KL. This version of F-PABEE adopts the KL divergence from probability mass distribution p^l to distribution p^{l+1}:

$$KL^{(l,l+1)} = \sum_{j=1}^{k} p_j^l log(p_j^{l+1}); \quad (5)$$

- F-PABEE-ReKL. It also adopts the KL divergence, but from the reversed direction, that is, the KL divergence from p^{l+1} to p^l:

$$KL^{(l+1,l)} = \sum_{j=1}^{k} p_j^{l+1} log(p_j^l) \quad (6)$$

- F-PABEE-SymKL. Note that the usual KL divergence is actually asymmetrical, and we can make it symmetrical with following equation:

$$SymKL = KL^{(l,l+1)} + KL^{(l+1,l)}; \quad (7)$$

- F-PABEE-JSD. This version of F-PABEE employs the Jenson-Shannon divergence:

$$JSD^{(l,l+1)} = 1/2(KL(p^l/\tilde{p}) + KL(p^{l+1}/\tilde{p})), \quad (8)$$

where $\tilde{p} = 0.5 * (p^l + p^{l+1})$ is the average distribution of p^l and p^{l+1}. Note that JSD is also a symmetrical divergence measure.

3.4 Similarity measures for MLC tasks

For MLC tasks, the judgment criterion based on Patience requires the prediction results of each label in the adjacent layer to be the same, which is too strict. Many results are correct on most labels, and wrong results on a few labels will be excluded, leading to difficulties in exiting.

F-PABEE can compare the prediction results, allowing a few different labels between different layers to achieve a balance between accuracy and efficiency. Therefore, our method also has significant advantages in multi-label.

For MLC tasks, we transform it into multiple binary classification problems and sum the divergences of all categories, and the formula is:

- F-PABEE-KL:

$$KL^{(l,l+1)} = \sum_{j=1}^{k}\sum_{i=1}^{2} p_{ji}^{l} log(p_{ji}^{l+1}); \quad (9)$$

- F-PABEE-ReKL:

$$KL^{(l+1,l)} = \sum_{j=1}^{k}\sum_{i=1}^{2} p_{ji}^{l+1} log(p_{ji}^{l}) \quad (10)$$

- F-PABEE-SymKL:

$$BiKL = KL^{(l,l+1)} + KL^{(l+1,l)}; \quad (11)$$

- F-PABEE-JSD:

$$JS^{(l,l+1)} = 1/2\sum_{j=1}^{k}\sum_{i=1}^{2}(KL(p_{ji}^{l}/\tilde{p}) + KL(p_{ji}^{l+1}/\tilde{p})), \quad (12)$$

where $\tilde{p} = 0.5 * (p_{ji}^{l} + p_{ji}^{l+1})$ is the average distribution of p^l and p^{l+1}.

3.5 Training

The model's training process is as follows: when the input sample is received, each output will calculate the loss according to its prediction result, and the sum of the losses will be optimized simultaneously for all outputs. Our model is trained and inferred on both SLC and MLC tasks, where the activation and loss functions are different for both tasks. We use the sigmoid activation function for the MLC function and calculate the loss using the cross-entropy function for binary classification. While, for the SLC tasks, we use the softmax function as the activation function and calculate the cross-entropy function according to the tasks.

After that, we optimize the model parameters by minimizing the overall loss function L ,which is the weighted average of the loss terms from all exits

$$L = \sum_{j=1}^{n} jL_j / \sum_{j=1}^{n} j \quad (13)$$

4 Experiments

4.1 Tasks and Datasets

We evaluated our proposed F-PABEE to the SLC tasks on the GLUE benchmark. We excluded STS-B since it is a regression task, and we also exclude the WNLI task follwing previous work. (Devlin et al., 2018) .Meanwhile, for MLC tasks, we evaluated on four datasets : MixATS (Hemphill et al., 1990) , MixSNLPS (Coucke et al., 2018) , AAPD (Yang et al., 2018) , and Stack overflow datasets. the size of the dataset is shown in the table 2 in the Appendix.

4.2 Baselines

We compare our F-PABEE with four groups of baselines.

Backbone models: We mainly choose the BERT-base and ALBERT-base model open-sourced by (Devlin et al., 2018) as the backbone model.

Budgeted exiting: If the computational budget is known, Budgeted exiting method are appointed a suitable exit layer of BERT, to predict all queries. we compare the model with different Budgeted exiting layers: Budgeted-Exit-3L, Budgeted-Exit-6L, Budgeted-Exit-9L.

Dynamic exiting: We compare our methods with a seriese of dynamic exiting methods, including Brachy Net, Shallow-Deep, BERxiT and PABEE.

4.3 Evaluation of early exiting method

For SLC tasks, we follow the GLUE benchmark to report the performances metrics on each task. For MLC tasks, we use the accuracy to evaluate the performance of models. The efficiency metric follows PABEE (Zhou et al.),we mainly evaluate the speedup ratio. Assuming the model has N layers, for each instances $x_i (i \in 0, 1, ..., N)$, the exiting layer is m_i, the average speedup ratio is :

$$Speedup = 1 - \sum_{1}^{N} m_i / \sum_{1}^{N} M \quad (14)$$

	CoLA		MNLI		MRPC		QNLI		QQP		RTE		SST-2	
	score	speedup	score	speedup	score	speedup	score	speedup	score	speedup	score	speedup	score	speedup
BERT base	0.542	0%	0.831	0%	0.868	0%	0.898	0%	0.892	0%	0.691	0%	0.913	0%
Budgeted-Exit-3L	0.0	75%	0.700	75%	0.758	75%	0.774	75%	0.818	75%	0.547	75%	0.810	75%
Budgeted-Exit-6L	0.0	50%	0.796	50%	0.847	50%	0.853	50%	0.893	50%	0.681	50%	0.886	50%
Budgeted-Exit-9L	0.519	25%	0.830	25%	0.870	25%	0.884	25%	0.903	25%	0.690	25%	0.912	25%
	0.0,	74%	0.638	76%	0.757	76%	0.742	80%	0.716	80%	0.547	76%	0.799	76%
BranchyNet	0.0	51%	0.783	53%	0.830	52%	0.871	47%	0.893	50%	0.674	47%	0.883	49%
	0.521	27%	0.830	25%	0.858	24%	0.893	27%	0.901	26%	0.680	26%	0.912	24%
	0.0	75%	0.641	77%	0.756	76%	0.743	78%	0.714	79%	0.547	76%	0.795	77%
Shallow-Deep	0.0	52%	0.782	51%	0.828	51%	0.872	49%	0.896	51%	0.672	48%	0.884	48%
	0.523	26%	0.829	26%	0.857	25%	0.893	26%	0.901	27%	0.678	26%	0.912	25%
	0.0	76%	0.635	76%	0.756	76%	0.733	78%	0.682	80%	0.553	77%	0.795	76%
BERxiT	0.1232	52%	0.784	51%	0.829	51%	0.870	48%	0.891	49%	0.673	47%	0.883	49%
	0.522	25%	0.832	26%	0.862	26%	0.896	27%	0.901	26%	0.681	27%	0.914	24%
	0.0	75%	0.639	77%	0.758	75%	0.736	81%	0.686	82%	0.558	75%	0.799	77%
PABEE	0.0	50%	0.789	52%	0.831	53%	0.872	46%	0.896	49%	0.677	46%	0.887	48%
	0.524	26%	0.834	24%	0.861	26%	0.898	28%	0.904	24%	0.683	28%	0.917	22%
	0.0	75%	0.669	72%	0.815	77%	0.762	75%	0.796	82%	0.560	76%	0.805	76%
F-PABEE	0.136	52%	0.839	53%	0.873	53%	0.886	54%	0.908	49%	0.681	47%	0.923	48%
	0.628	25%	0.842	28%	0.880	26%	0.909	27%	0.912	25%	0.689	25%	0.924	25%

Table 1: Experimental results of different early exiting methods with the same fine-tuned BERT backbone on the GLUE benchmark. The results show that F-PABEE is effective in accelerating BERT's inference with less performance loss compared with the baseline methods.

4.4 Experimental setting

Training: we perform the batch size of 16,32,128, and learning rate of 1e-5, 2e-5, 3e-5, 5e-5 with an Adam optimizer. We implement the base of Hugging face's Transformers on the development set. Moreover, we conduct our experiments on a single Nvidia TITAN X 24GB GPU.

Inference: We calculated the similarity score between adjacent layers for the dynamically early exit and the model early exit if the score is less than the defined threshold. In inference experiments, we set different patience and score thresholds for different tasks, where the patience is between 1-12, and the threshold is between 0-1.

5 Results and discussions

5.1 Overall comparison

In our experiments, We explore the performance of the F-PABEE model on SLC and MLC tasks, respectively. First, we analyze the performance of the model on the SLC tasks: we compare our F-PABEE with BERT-base, Budgeted-Exit-3L, Budgeted-Exit-6L, Budgeted-Exit-9L, BranchyNet, Shallow-deep, BERxiT, PABEE model among six classification tasks on the GLUE benchmark, and the Results is shown in Table 1, which demonstrate that F-PABEE outperforms existing early exit models on SLC tasks.

In addition, we plot the Accuracy-Speedup curves of the BERT and ALBERT-based models on the SLC and MLC task, among them, the SLC task is CoLA, and the MLC task is MixSNIPS, and the results are shown in Figure 2 and the results on other tasks are shown in Appendix. It can be seen that the efficiency and accuracy of F-PABEE are higher than those of PABEE and BERxit early exit methods. This indicates that our F-PABEE can balance accuracy and efficiency for simple and complex tasks by dynamically adjusting the Similarity score and Patience thresholds.

Moreover, we also compared the effects of four different similarity score calculation methods for the SLC and MLC task, among them, the SLC task is QNLI, and the MLC task is MixATIS. The results are shown in Figure 3 and the results on other tasks are shown in Appendix. This indicates that F-PABEE-JSD works better than other score calculation methods: F-PABEE-KL, F-PABEE-ReKL, and F-PABEE-SymKL.

Furthermore, we also explored the middle exit layer of the model on MRPC and MixSNIPS task. The results are shown in Figure 4. It can be found that when the speed-up ratio is low, most of the model exits in the middle layer. When the speed-up ratio is high, the model exits at layers 2-3 in most cases, which shows our model significantly improves the inference efficiency.

5.2 Analysis

Comparison between F-PABEE and Budgeted exiting methods: If the computational budget is known, Budgeted Exiting are appointed suitable exit layer of the model to predict all queries. our F-PABEE performs better than Budgeted exiting methods, which attributes to the facts that

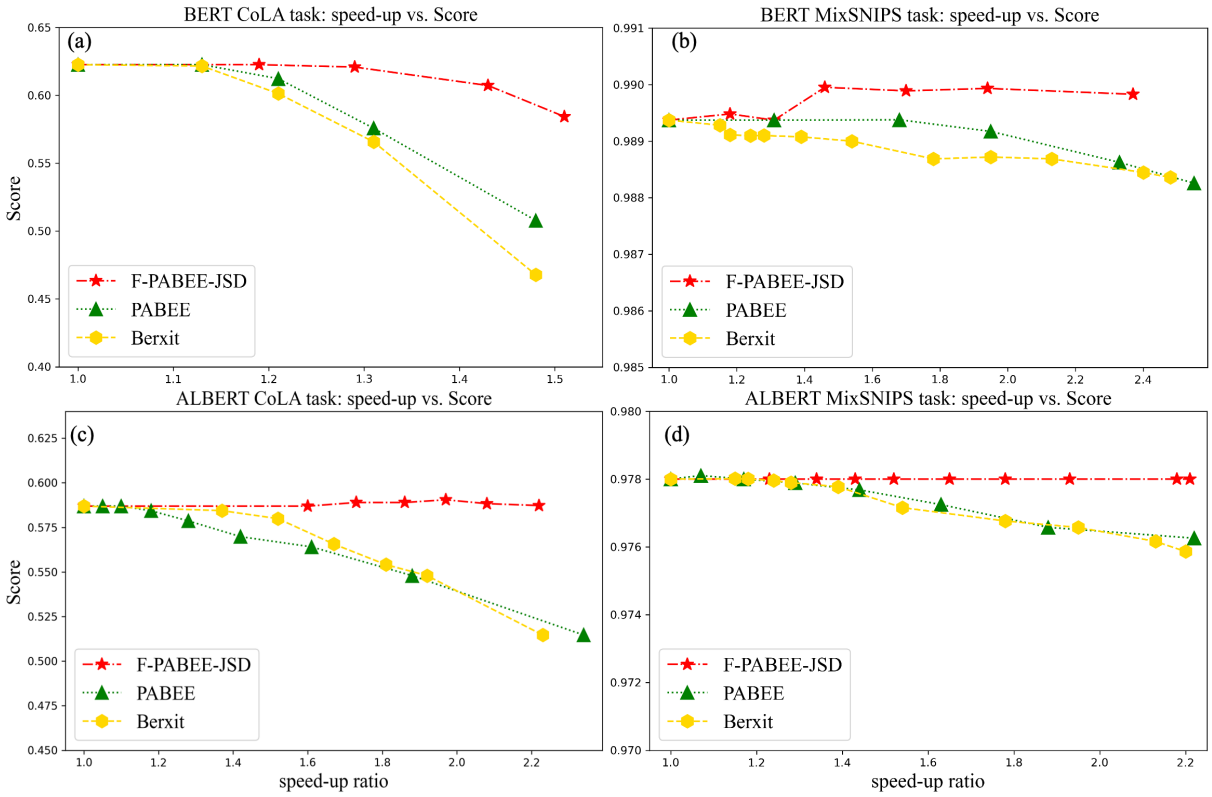

Figure 2: Speed-accuracy curves of F-PABEE, PABEE,BERxiT on SLC and MLC tasks

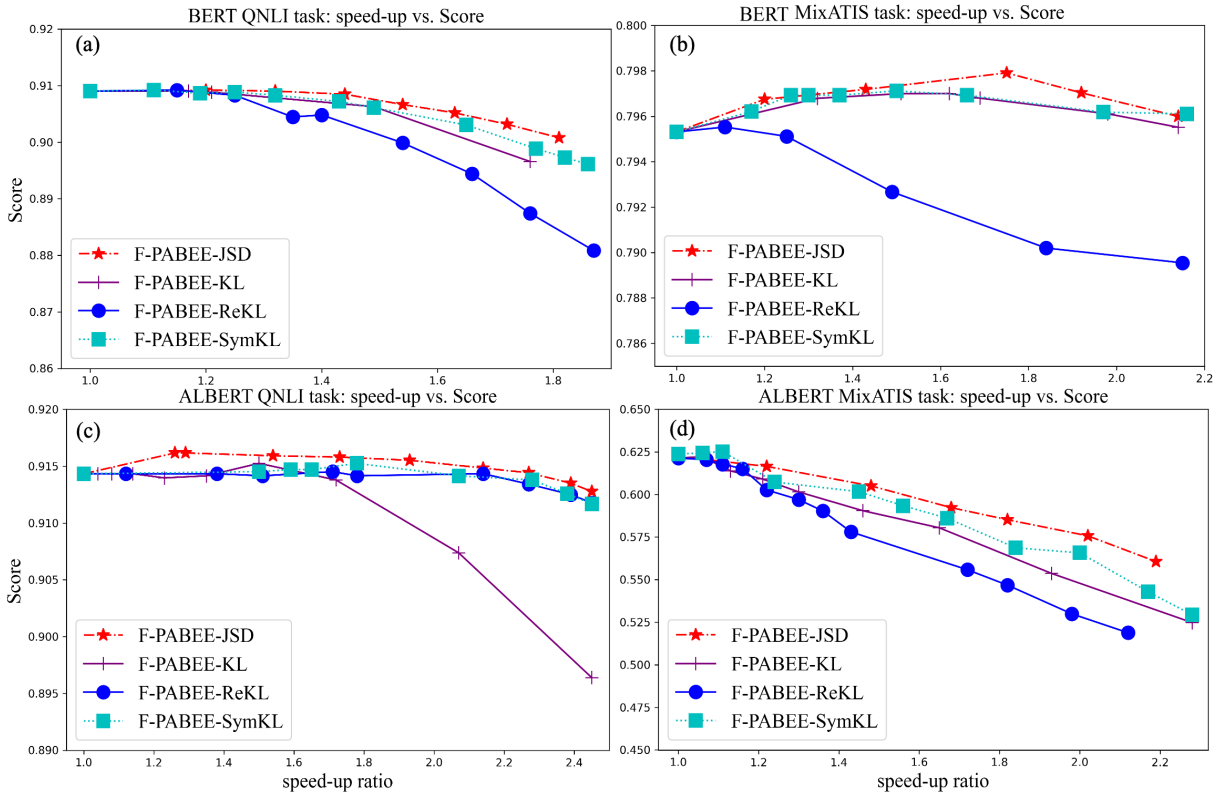

Figure 3: Speed-accuracy curves of different similarity score calculation methods on SLC and MLC tasks

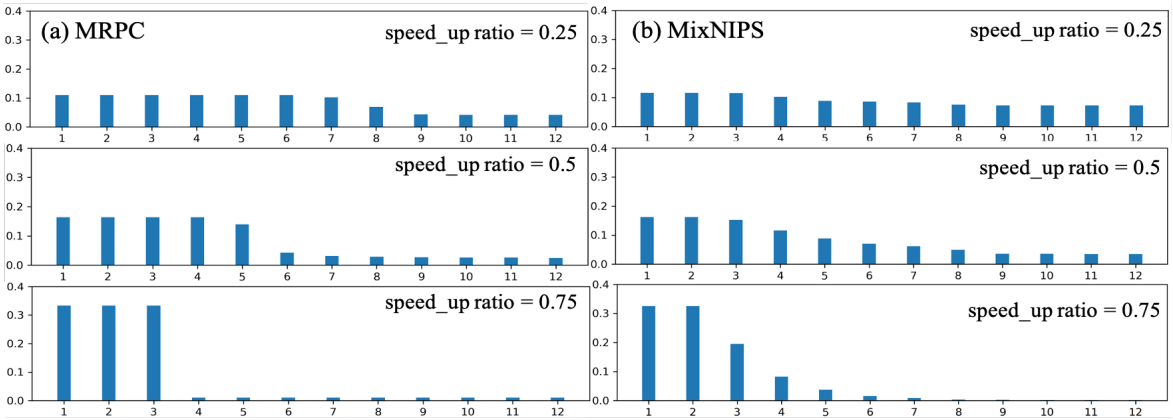

Figure 4: The distribution of executed layers of MRPC and MixSNIPS on average at three different speeds(0.25,0.5,0.75)

some simple samples go through shallow layers, while some difficult tasks may require deeper feature encoding from deeper BERT, however the exit layer of Budgeted exiting is limited. Therefore, the performance of Budgeted exiting methods is limited.

Comparison between F-PABEE and other adaptive early exit methods: It can be seen in Figure 2 that as the speedup ratio slowly increased, the accuracy of BERxiT dropped the most. We suspect it may be because the early exit is based on predicted probabilities nor the inner agreement between earlier and later internal classifiers. It leads to more samples meeting the early exit criteria by chance before understanding the whole sentence's deep meaning and reaching a steady state. The patience-based method PABEE dynamically exits early based on the internal agreement between adjacent internal classifiers. However, with the same accuracy, its speedup ratio is not as high as F-PABEE, which may be because the exit criteria of PABEE are strict, which leads to a drop in efficiency. Furthermore, when the patient is too high, the later internal classifier may suffer from the overthinking problem and make a wrong prediction, breaking the stable state built by previous internal classifiers. Compared with the above methods, F-PABEE can flexibly adjust the hyper parameters according to the data and request traffic, which can obtain the local information of sentences efficiently and understand the deep meaning of sentences, avoiding the problem of overthinking and inefficiency.

Impact of different similarity score calculation methods: For ablation study, the reason why F-PABEE-JSD performs better than other similarity score calculation methods such as F-PABEE-KL, F-PABEE-ReKL, and F-PABEE-SymKL is analyzed. We suspect it may because the KL-divergence is asymmetric, not an accurate distance measure, and has a wide range, which increases the difficulty to tuning the hyper parameters. F-PABEE-JSD introduces JSD-divergence based on KL-divergence, which is a symmetric divergence, and the discrimination of similarity is more accurate than asymmetric divergence. Moreover, its divergence range is between 0-1, making the hyper parameters easier to tune. Therefore, F-PABEE-JSD outperforms other early exit methods.

6 Conclusion

In this paper, we proposed F-PABEE, a novel and efficient method combining the PABEE with a softer cross-layer comparison strategy, which is more flexible than the previous state-of-the-art early exiting method PABEE since it can achieve different speed-performance tradeoffs by adjusting the similarity score threshold and the patience parameter. Divergence-based metrics like KL-divergence and JSD-divergence measure the similarity of predicted probability distributions from the current and previous layers. Extensive experiments on SLC and MLC tasks demonstrate that:(a) our F-PABEE performs better than the previous SOTA adaptive early exit methods for SLC and MLC tasks. (b) F-PABEE performs well on different PLMs such as ALBERT. (c) ablation studies show that JSD-divergence's similarity calculation method outperforms other similarity score calculation methods for our F-PABEE model.

7 Ethical considerations

F-PABEE can be seen as a natural extension of the PABEE method. It is more flexible than PABEE since it can adjust the similarity score and the patience threshold according to the data characteristics and request traffic. Moreover, it is more applicable for pre-trained models to be deployed on real-time search engines or software for public communication and edge devices with limited hardware, such as mobile phones and smartwatches. Furthermore, It does not introduce new ethical concerns, but more work is needed to determine its effect on biases encoded in PLMs.

8 Limitation

Although F-PABEE can improve both efficiency and accuracy, it also has limitations. For example, this method is only suitable for classification problems, not regression problems, which restricts the application of F-PABEE. Moreover, more suitable similarity score calculation methods may exist that can improve efficiency. Therefore, for future work, we would like to explore efficient models for regression tasks and more similar evaluation metrics for more tasks and models.

References

Alice Coucke, Alaa Saade, Adrien Ball, Théodore Bluche, Alexandre Caulier, David Leroy, Clément Doumouro, Thibault Gisselbrecht, Francesco Caltagirone, Thibaut Lavril, Maël Primet, and Joseph Dureau. 2018. Snips voice platform: an embedded spoken language understanding system for private-by-design voice interfaces. *arXiv*, abs/1805.10190.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. 2018. Universal transformers. *arXiv*, abs/1807.03819.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: pre-training of deep bidirectional transformers for language understanding. *ArXiv*, abs/1810.04805.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. 2019. Depth-adaptive transformer. *arXiv*, abs/1910.10073.

Angela Fan, Edouard Grave, and Armand Joulin. 2019. Reducing transformer depth on demand with structured dropout. *arXiv*, abs/1909.11556.

Charles T. Hemphill, John J. Godfrey, and George R. Doddington. 1990. The ATIS spoken language systems pilot corpus. In *Speech and Natural Language: Proceedings of a Workshop Held at Hidden Valley, Pennsylvania, June 24-27,1990*.

Lu Hou, Lifeng Shang, Xin Jiang, and Qun Liu. 2020. Dynabert: Dynamic BERT with adaptive width and depth. *arXiv*, abs/2004.04037.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2019. Tinybert: Distilling BERT for natural language understanding. *arXiv*, abs/1909.10351.

Yigitcan Kaya and Tudor Dumitras. 2018. How to stop off-the-shelf deep neural networks from overthinking. *arXiv*, abs/1810.07052.

Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2021. I-BERT: integer-only BERT quantization. *arXiv*, abs/2101.01321.

Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2019. ALBERT: A lite BERT for self-supervised learning of language representations. *arXiv*, abs/1909.11942.

Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. 2021. A survey of transformers. *arXiv*, abs/2106.04554.

Weijie Liu, Peng Zhou, Zhe Zhao, Zhiruo Wang, Haotang Deng, and Qi Ju. 2020. Fastbert: a self-distilling BERT with adaptive inference time. *arXiv*, abs/2004.02178.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized BERT pretraining approach. *arXiv*, abs/1907.11692.

Paul Michel, Omer Levy, and Graham Neubig. 2019. Are sixteen heads really better than one? *arXiv*, abs/1905.10650.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv*, abs/1910.01108.

Roy Schwartz, Gabi Stanovsky, Swabha Swayamdipta, Jesse Dodge, and Noah A. Smith. 2020. The right tool for the job: Matching model and instance complexities. *arXiv*, abs/2004.07453.

Siqi Sun, Yu Cheng, Zhe Gan, and Jingjing Liu. 2019. Patient knowledge distillation for BERT model compression. *arXiv*, abs/1908.09355.

Surat Teerapittayanon, Bradley McDanel, and H.T. Kung. 2016. Branchynet: Fast inference via early exiting from deep neural networks. In *2016 23rd International Conference on Pattern Recognition (ICPR)*, pages 2464–2469.

Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. 2020. Deebert: Dynamic early exiting for accelerating BERT inference. *arXiv*, abs/2004.12993.

Ji Xin, Raphael Tang, Yaoliang Yu, and Jimmy Lin. 2021. BERxiT: Early exiting for BERT with better fine-tuning and extension to regression. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 91–104, Online. Association for Computational Linguistics.

Pengcheng Yang, Xu Sun, Wei Li, Shuming Ma, Wei Wu, and Houfeng Wang. 2018. SGM: sequence generation model for multi-label classification. *arXiv*, abs/1806.04822.

Zhilin Yang, Zihang Dai, Yiming Yang, Jaime G. Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. 2019. Xlnet: Generalized autoregressive pretraining for language understanding. *arXiv*, abs/1906.08237.

Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian J. McAuley, Ke Xu, and Furu Wei. BERT loses patience: Fast and robust inference with early exit. *arXiv*.

A Appendix

A.1 Details of the datasets

Here we presented the in table 2 the details of the datasets for both SLC and MLC tasks.

Table 2: Statistics of single and multi label datasets

Dataset	Labels	Train/Dev/Test
COLA	2	8.5k/1.0k/1.0k
RTE	2	2.5k/0.3k/3.0k
MRPC	2	3.7k/0.4k/1.7k
SST-2	2	67k/0.9k/1.8k
QNLI	2	105k/5.5k/5.5k
MNLI	3	393k/9.8k/9.8k
AAPD	54	53k/6.7k/6.7k
Stackflow	50	80k/10k/10k
MixATS	17	18k/1k/1k
MixSNLPS	7	45k/2.5k/2.5k

A.2 comparson of different adaptive early exit methods

In the main context, we present the Speed-accuracy curves for SLC task CoLA and MLC task MixSNIPS, and we present the Speed-accuracy curves of other SLC and MLC tasks as shown in Figure. 5 and Figure. 6.

A.3 comparison of different similarity score calculation methods

In the main context, we present the ablation results for different similarity calculation methods of SLC task CoLA and MLC task MixSNIPS. We also present the ablation results of other SLC and MLC tasks here. as shown in Figure. 7 and Figure. 8.

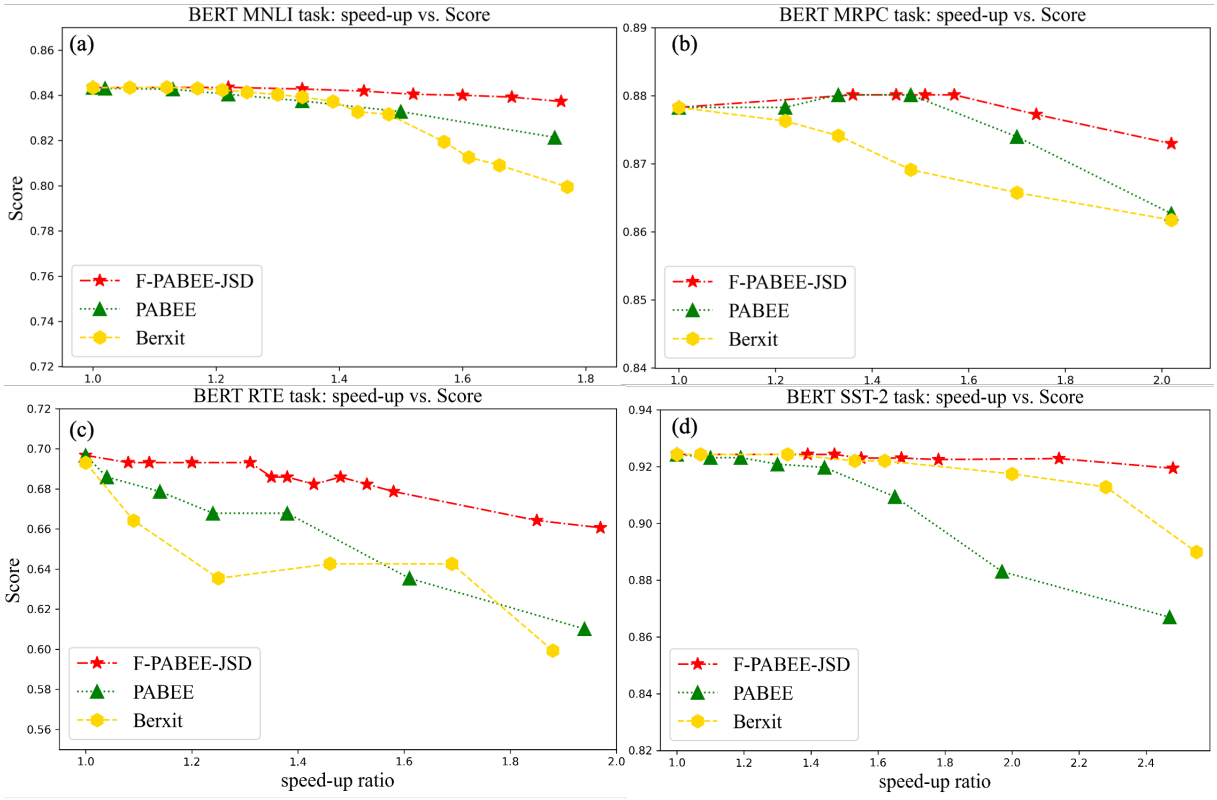

Figure 5: Speed-accuracy curves of F-PABEE, PABEE,BERxiT on SLC tasks with BERT as the backbone

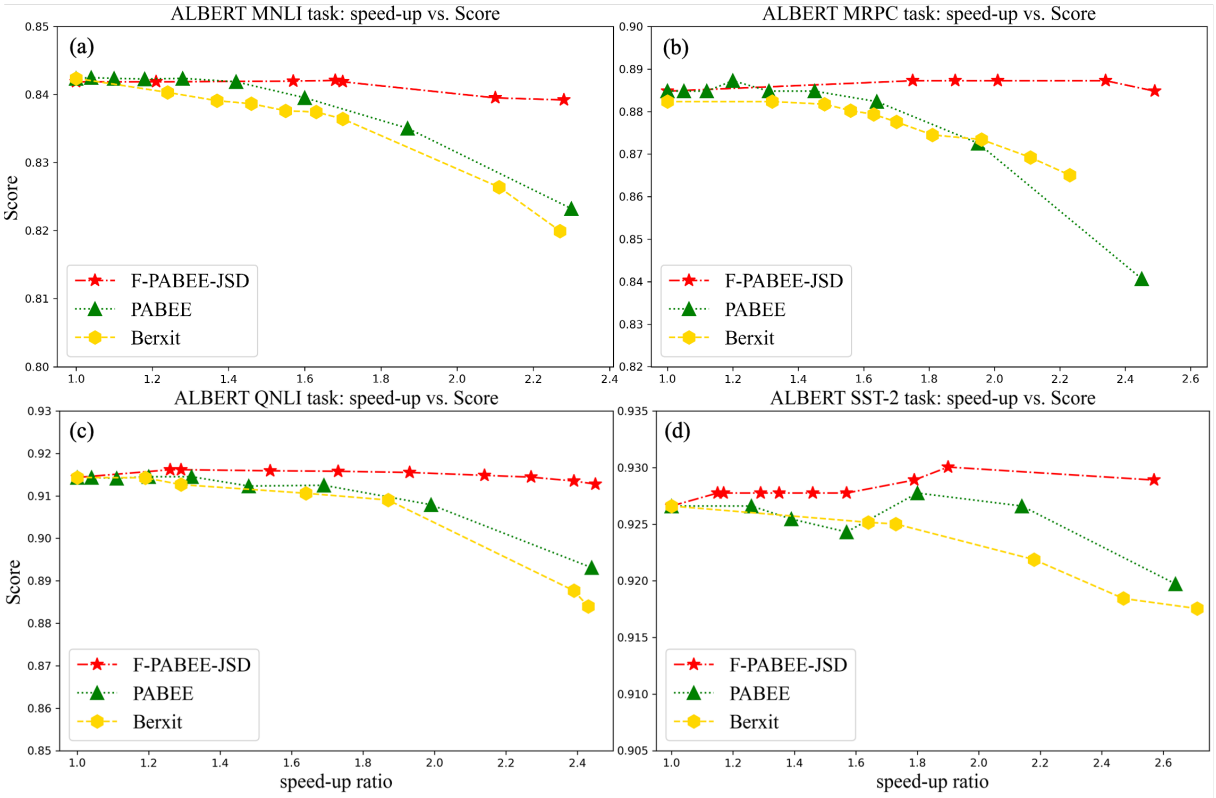

Figure 6: Speed-accuracy curves of F-PABEE, PABEE,BERxiT on SLC tasks with ALBERT as the backbone

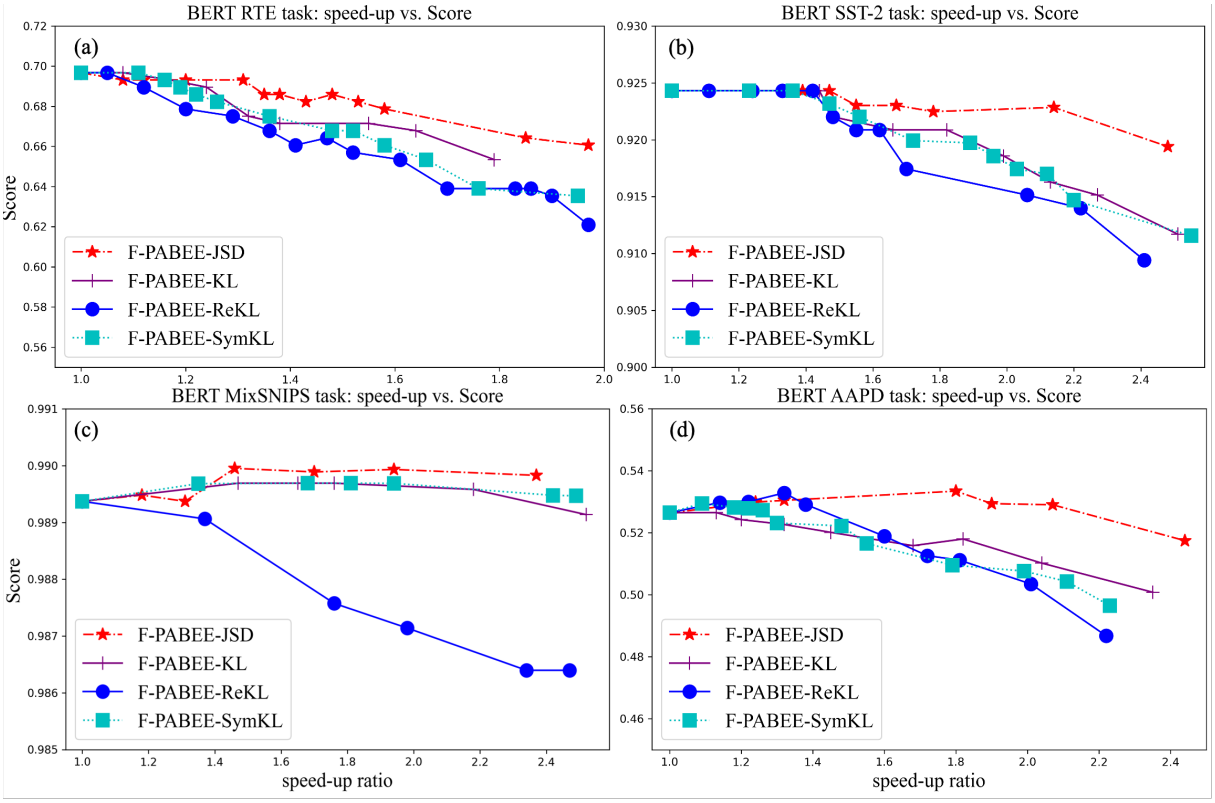

Figure 7: Speed-accuracy curves of different similarity score calculation methods on SLC and MLC tasks with BERT as backbone

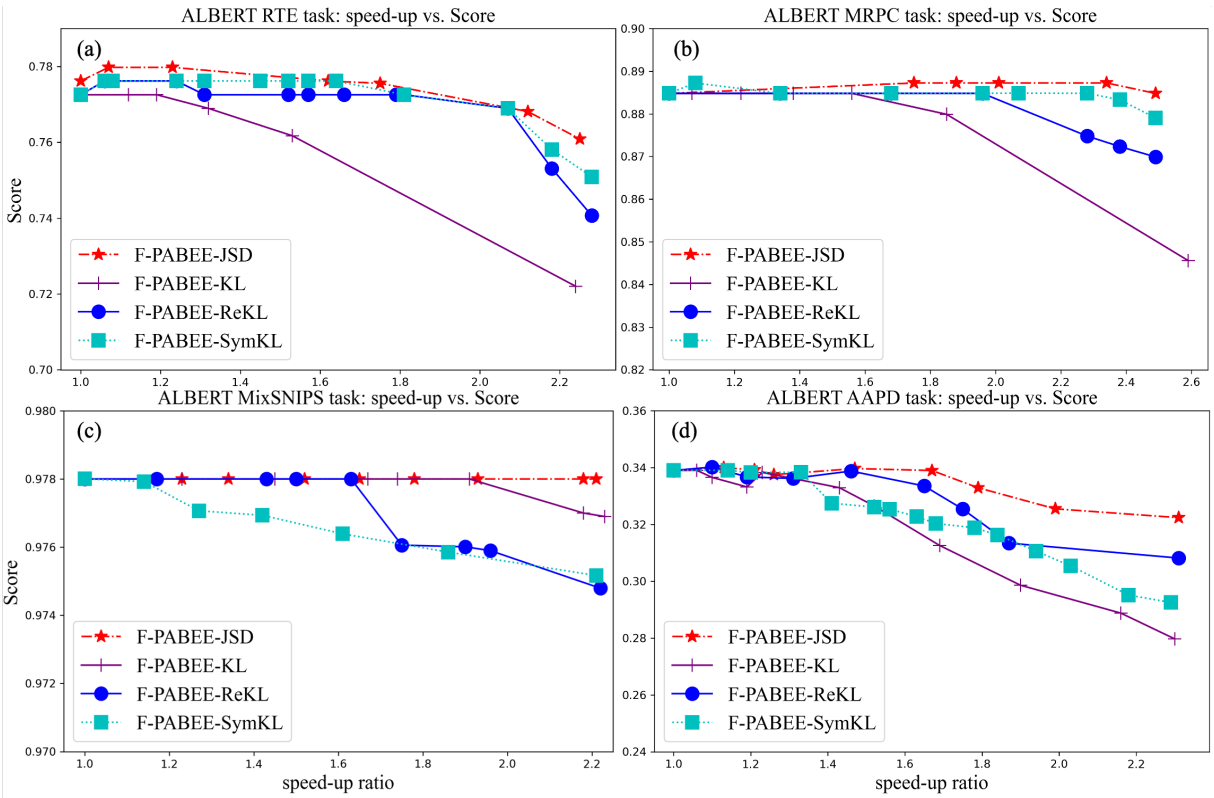

Figure 8: Speed-accuracy curves of different similarity score calculation methods on SLC and MLC tasks with ALBERT as backbone